Memory Test Strategy (iOS NE ceiling + leak soak)

Memory Test Strategy (iOS NE ceiling + leak soak)

Revision: 2 **Last modified:** 2026-06-26T12:00:00Z

> **Reconciled (§11.4.35, 2026-06-26):** the 24 h MEM-SOAK-LEAK / SLO4 single run is now an **explicit, documented §11.4.50 carve-out recorded at the gate (§6)** — verdict deterministic-by-construction (re-run on any regression), not a silent deviation; only the short G3 probe is N=3. §5 + §6 state the carve-out, replacing the prior bare prose.

> Master technical specification — Volume 8 (Testing & QA), nano-detail document **memory**, one of the seven §11.4.169 cross-cutting test-type deep-dives. It deepens `10-testing-acceptance-and-qa.md` §5.12 (MEM) into an implementation-ready bank for the **make-or-break** iOS `NEPacketTunnelProvider` memory ceiling (Phase-0 gate G3), 24 h leak soaks, and the coordinator's bounded memory at 10k streams (SLO4). SPEC-ONLY: it describes harness, fixtures, evidence, gate, and the paired §1.1 mutation; it does not build the product. Sources cited inline by id — `[OVERVIEW]` = doc 10; `[SYNTHESIS]` = `v09-research/_SYNTHESIS.md`; `[04_P0]` = Phase-0 spike; `[reconcile]` = [`../v03-control-plane/reconciliation-flow.md`]; `[TM]` = [`../v05-security/threat-model.md`]. Claims not grounded in the evidence base are marked **UNVERIFIED** per constitution §11.4.6 — never fabricated.

Table of contents

0. Scope on HelixVPN surfaces — why memory is make-or-break

The single decision the whole client architecture rests on is the **iOS `NEPacketTunnelProvider` memory ceiling** (~15 MB historical [SYNTHESIS §5]): iOS jetsam-kills a packet-tunnel extension that exceeds its footprint, so the entire choice to write `helix-core` in **Rust, not Go** (decision D2 [SYNTHESIS §3]) is justified *only* if the Rust core + WG + transport stack stays under that ceiling with ≥30% headroom on a real device. The Phase-0 gate **G3** is the make-or-break test for the program: if Rust cannot meet it, the architecture changes [04_P0]. This bank also covers leak-freedom over a 24 h soak (the reconcile loop and the QUIC/MASQUE buffer pool must not grow), Android `VpnService` RSS, and the coordinator's bounded memory at 10k streams (SLO4).

Surface	Memory claim	Why it matters	Gate
iOS `NEPacketTunnelProvider`	peak `phys_footprint` < ceiling, ≥30% headroom	jetsam kills the tunnel above the ceiling	**G3 (make-or-break)**
helix-core 24 h soak	no RSS growth over sustained transfer	a leak in reconcile/QUIC buffers eventually OOMs the extension	MEM soak
Android `VpnService`	RSS within the Android background budget	a too-heavy service is killed under memory pressure	MEM
coordinator @ 10k streams	bounded RSS over a 24 h soak	an unbounded per-stream allocation OOMs the gateway	**SLO4**

1. The four memory claims

(A) G3 — iOS NE peak footprint (make-or-break). On a **real iOS device** (the simulator does not reproduce the jetsam ceiling, decision QA-D3 — overview §10), the Rust core under sustained real traffic must keep `task_vm_info.phys_footprint` (the metric iOS jetsam actually evaluates, **not** resident-size) below the ceiling with **≥30% headroom** (peak < ceiling × 0.7). This is the program's go/no-go.

(B) Leak-freedom over a 24 h soak. A sustained real transfer for 24 h must show **no monotonic RSS growth** — a leak in the reconcile loop (each map apply must free the old peer set) or the QUIC/MASQUE buffer pool (buffers must be returned, not accumulated) would be a slow OOM. The claim is a flat (within jitter) RSS-vs-time series, asserted by a monotonic-growth detector with a slope threshold.

(C) Android `VpnService` RSS. The Android tunnel service RSS (via `dumpsys meminfo <pkg>`) stays within the platform background budget so Android does not kill it under memory pressure.

(D) SLO4 — coordinator bounded memory at 10k streams. N=10k simulated agents holding `WatchNetworkMap` streams must keep coordinator RSS **bounded** over a 24 h soak — per-stream state is O(1) and the in-memory topology graph is shared, not copied per stream ([reconcile], [TM `T-COORD-D-1`]). An unbounded per-stream allocation is the OOM vector.

---

2. Harness — per-platform RSS samplers

Memory is platform-specific (§11.4.81 cross-platform parity); each platform has its OS-correct sampler, chosen at runtime. **No mocks** — the metric must come from the real OS.

Platform	Sampler	Metric (the jetsam/ OOM-relevant one)
iOS NE	`task_info(TASK_VM_INFO)` → `phys_footprint`; Instruments `Allocations`/`Leaks` `.trace`	`phys_footprint` (the jetsam metric, not resident size)
Linux (helix-core, edge, coordinator)	`/proc/<pid>/status` `VmRSS` / `smaps_rollup` Pss	RSS / Pss over time
Android `VpnService`	`dumpsys meminfo <pkg>` (TOTAL PSS)	PSS in the background budget
coordinator soak	`/proc/<pid>/status` at 1-min cadence over 24 h	RSS bounded / no growth

```swift
// shims/apple/PacketTunnelProvider+MemoryProbe.swift — G3
        evidence emitter (overview §5.12)
func sampleFootprint() -> UInt64 {                  // bytes;
        appended to qa-results/mem/ios_rss.csv
    var info = task_vm_info_data_t()
    var count =
        mach_msg_type_number_t(MemoryLayout<task_vm_info>.size /
        MemoryLayout<integer_t>.size)
    let kr = withUnsafeMutablePointer(to: &info) {
        $0.withMemoryRebound(to: integer_t.self, capacity:
        Int(count)) {
            task_info(mach_task_self_,
        task_flavor_t(TASK_VM_INFO), $0, &count) } }
    return kr == KERN_SUCCESS ? info.phys_footprint : 0   //
        phys_footprint == the jetsam metric
}
```

The samplers themselves must be light (Heisenberg constraint, §11.4.24-class): the probe samples at a fixed cadence (≤1 Hz) and writes a CSV out-of-band; it must not itself perturb the footprint it measures. The bank runs under `make test` MEM stage for short probes and a backgrounded 24 h soak job (§11.4.89). Coordinator soak boots PG+Redis via `containers` (§11.4.76, rootless §11.4.161).

---

3. Fixtures — real device, real soak (§11.4.27)

Fixture	What	Why real
real iOS device (paired)	a physical iPhone running the NE build	the jetsam ceiling does **not** reproduce in the simulator (QA-D3)
`device_build.ipa`	the shipping NE extension build (release config)	G3 must measure the *shipping* artifact (§11.4.108), not a debug build with allocator debug overhead
sustained real transfer	a real `iperf3`/file transfer through the tunnel	a leak shows only under real buffer churn, not an idle tunnel
10k agent-fuzzer	a real load generator opening 10k `WatchNetworkMap` streams	SLO4 needs real streams holding real per-stream state

If **no real iOS device is available**, G3 is an honest §11.4.3 `SKIP: hardware_not_present` with a tracked operator-attended migration item (QA-D3, overview §10) — **never** a simulator PASS faked as a device result (that would be a B3 wrong-plane bluff).

---

4. Captured evidence (§11.4.69 / .107)

Every PASS cites artifacts under `qa-results/mem/<run-id>/`. The §11.4.69 class is `MEM`. §11.4.107 liveness applies: the RSS series must be over a **window** during *real, advancing* traffic (a single footprint sample while the tunnel is idle is not proof — the core must be doing real work).

Test	Artifact	Asserts
G3-IOS-PEAK	`ios_rss.csv` + Instruments `.trace`	peak `phys_footprint` < ceiling × 0.7 (≥30% headroom)
MEM-SOAK-LEAK	`core_rss_24h.csv`	no monotonic growth (slope < threshold) over 24 h
MEM-ANDROID-RSS	`android_meminfo.csv`	PSS within the background budget

Test	Artifact	Asserts
SLO4-COORD-BOUNDED	`coord_rss_24h.csv` + `stream_count.csv`	RSS bounded while holding 10k streams over 24 h

The evidence is the **time series**, not a point: a flat-or-bounded RSS-vs-time curve under load. A single in-bounds sample is a B-class point-not-window bluff (§11.4.107(1)); the analyzer asserts the *peak over the window* and the *growth slope*, both self-validated (§5, golden-good flat series PASSes, golden-bad leaking series FAILs).

---

5. Determinism (§11.4.50)

G3 short probe — N=3 (the §11.4.50 default).
`ab_run_n_times "g3-ios-peak" 3 run_ios_probe` runs the short G3 peak-RSS probe 3× against the same `.ipa` MD5 + same device; the evidence-hash is over `(peak_footprint_bucket, headroom_pct_meets_30: bool)` — all 3 MUST agree. A run where the headroom meets 30% in 2 of 3 is **auto-FAIL** (the ceiling margin must be reliable, not lucky).

24 h soak (MEM-SOAK-LEAK + SLO4) — an explicit, documented §11.4.50 carve-out. A 24 h run is **not** repeated N=3 (three back-to-back 24 h soaks per gate is operationally infeasible). The carve-out is justified because the soak's verdict is **deterministic-by-construction**, not deterministic-by-repetition: `peak < ceiling` and `no monotonic RSS growth` are falsifiable facts about *that one authoritative run*, and the run is **re-run on any regression** (a divergent slope across two soaks is a regression triggering the §11.4.4 STOP). Running the soak once does **not** violate §11.4.50 — the property under test is a per-run threshold / monotonicity verdict, not a flake-prone pass/fail the N-iteration loop exists to catch. This carve-out is recorded **at the gate (§6)**, never left as bare prose.

---

6. Acceptance gate

Gate	Bar	Evidence	Phase
G3 (make-or-break)	peak `phys_footprint` < ceiling, **≥30% headroom**, on a real device	`ios_rss.csv` + `.trace`	**Phase 0 (release-blocking go/no-go)**
MEM-SOAK-LEAK	no monotonic RSS growth over 24 h under load	`core_rss_24h.csv`	MVP
MEM-ANDROID-RSS	PSS within background budget	`android_meminfo.csv`	MVP
SLO4	coordinator RSS bounded @ 10k streams / 24 h	`coord_rss_24h.csv`	MVP (release-blocking SLO)

G3 is the program's make-or-break gate: a No-Go means the architecture changes (Rust→? or a memory budget rework), escalated per §11.4.66 — never silently overrun [04_P0]. G3 + SLO4 are §11.4.132 risk-ordered high (G3 is the highest-risk Phase-0 gate). The MEM cell appears in the ledger for F-IOS-NE-MEM (G3) and SLO4 (overview §6.3, §7.2).

> **§11.4.50 carve-out (recorded at the gate, per §5).** The **24 h** rows — `MEM-SOAK-LEAK` and `SLO4` — are NOT run N=3: a 24 h soak is run **once** as the authoritative run, its verdict deterministic-by-construction (`peak < ceiling` / `no monotonic growth`) and **re-run on any regression**. Only the short **G3** probe is run N=3 (`ab_run_n_times`, §5). This is the explicit, documented carve-out — not a silent §11.4.50 deviation.

---

7. The paired §1.1 mutation

```
MUTATION (paired §1.1, gate CM-MEM-LEAK-DETECTOR-SELFVALIDATED):
  Replace the golden-bad leaking RSS series fixture with a flat
one (or
  loosen the monotonic-growth slope threshold to accept any
slope).
EXPECTED:  the leak-detector self-validation test that asserts the
           golden-bad (leaking) series FAILs now PASSES the leak →
           meta-test FAILs → mutation caught (defeats B5 — an
analyzer
           that passes its golden-bad fixture is the bluff).
RESTORE:   re-instate the leaking golden-bad fixture / strict
slope; re-run → GREEN.
```

A second mutation (`CM-SLO4-BOUNDED`) introduces a per-stream allocation that is never freed in the coordinator fan-out; expected: `coord_rss_24h.csv` shows unbounded growth at 10k streams → SLO4 FAILs → caught (this is the real OOM vector the gate guards). A third (`CM-G3-METRIC-IS-FOOTPRINT`) swaps the G3 metric from `phys_footprint` to resident-size (which under-reports the jetsam-relevant footprint); expected: the metric-correctness check FAILs because the asserted metric no longer matches the jetsam metric (defeats a B3 wrong-plane bluff). All mutations restored, tree verified quiescent (§11.4.84) before commit.

---

8. Test skeletons

```
# rig/ios_g3_probe.sh — G3-IOS-PEAK on a REAL device (QA-D3),
        §11.4.108 shipping artifact
set -euo pipefail
out="qa-results/mem/g3_$(date +%s)"; mkdir -p "$out"
device_serial=$(ios_paired_device) || ab_skip_with_reason "G3 iOS
        peak" hardware_not_present  # §11.4.3, never a sim PASS
ios_install "$device_serial" device_build.ipa
        # the RELEASE NE build
ios_start_tunnel "$device_serial"
ios_run_sustained_transfer "$device_serial" 600 &
        # 10 min of REAL traffic (liveness §11.4.107)
```

```
ios_sample_footprint_1hz "$device_serial" "$out/ios_rss.csv"
        # phys_footprint series
ios_capture_instruments_trace "$device_serial" "$out/leaks.trace"
peak=$(max_col "$out/ios_rss.csv" phys_footprint)
ceiling=$NE_CEILING_BYTES
awk -v p="$peak" -v c="$ceiling" 'BEGIN{exit !(p < c*0.7)}' \
  && ab_pass_with_evidence "G3: peak $peak < ceiling $ceiling
        (>=30% headroom)" "$out" \
  || ab_fail "G3 NO-GO: peak $peak exceeds 70% of ceiling
        $ceiling — escalate (§11.4.66)"

# scripts/mem_soak.sh — MEM-SOAK-LEAK 24 h, no monotonic growth
        (backgrounded §11.4.89)
set -euo pipefail
out="qa-results/mem/soak_$(date +%s)"; mkdir -p "$out"
trap 'stop_transfer; rig/netns_down.sh' EXIT
start_sustained_transfer &
sample_rss_1min "$(core_pid)" "$out/core_rss_24h.csv" 86400
        # 24 h at 1-min cadence
slope=$(monotonic_growth_slope "$out/core_rss_24h.csv")
        # bytes/hour
within_threshold "$slope" \
  && ab_pass_with_evidence "no RSS leak over 24h (slope=$slope B/
        h)" "$out" \
  || ab_fail "RSS leak: slope=$slope B/h exceeds threshold —
        §11.4.4 STOP + systematic-debug"

// helix-go/internal/coordinator/slo4_soak_test.go — SLO4 bounded
        @ 10k streams
func TestCoordinatorMemoryBoundedAt10kStreams(t *testing.T) {
    infra := mustBootRedisPG(t)                          //
        containers, rootless §11.4.161
    coord := startCoordinator(t, infra)
    agents := openNStreams(t, coord, 10_000)             // 10k
        real WatchNetworkMap streams
    rss := sampleRSSOver(t, coord.Pid(), 24*time.Hour,
        time.Minute)
    requireBounded(t, rss, slo4Ceiling)                  // RSS
        bounded, no unbounded per-stream growth
    writeEvidence(t, "qa-results/mem/coord_rss_24h.csv", rss)
    _ = agents
    // paired §1.1 (CM-SLO4-BOUNDED): leak a per-stream allocation
        → unbounded growth → SLO4 FAILs
}
```

Honest boundary (§11.4.6). The exact NE ceiling value (`NE_CEILING_BYTES`) is a platform constant cited as ~15 MB historical [SYNTHESIS §5] but is **UNVERIFIED** for the current iOS version on the target device class until the G3 probe runs on the real device — the ceiling is iOS-version and device-dependent, and the spec uses the conservative historical figure. The 24 h soak proves no leak *over that window* under *that workload*; a leak with a period longer than 24 h, or under a workload not exercised, is an §11.4.118 discovery gap, not a proven absence. SLO4's bound at exactly 10k streams is the MVP target; the convergence-and-memory behaviour beyond 10k is a Phase-2 SCALE concern ([TM `T-COORD-D-1`] notes the 10k-stream p99 is itself **UNVERIFIED** until the soak runs).

Sources verified

- [OVERVIEW] [`../10-testing-acceptance-and-qa.md`] — §5.12 (MEM strategy + `sampleFootprint` skeleton + real-device requirement), §6.3 (F-IOS-NE-MEM ledger), §7.1 (G3 gate), §7.2 (SLO4), §8 (determinism/risk-order), §10 QA-D3. (Read 2026-06-26.)
- [SYNTHESIS] `v09-research/_SYNTHESIS.md` — §3 (D2 Rust-not-Go decision), §5 (~15 MB NE ceiling). (Cited via overview.)
- [04_P0] `04_VPN_CLD/HelixVPN-Phase0-Spike.md` — G3 make-or-break iOS memory gate, ≥30% headroom. (Cited via overview.)
- [reconcile] [`../v03-control-plane/reconciliation-flow.md`] — coordinator topology graph + stream fan-out (SLO4 per-stream state). (Read 2026-06-26.)
- [TM] [`../v05-security/threat-model.md`] — `T-COORD-D-1` (10k-stream p99/memory **UNVERIFIED** soak). (Read 2026-06-26.)
- Constitution: §11.4.169, §11.4.27 (real device/no mocks), §11.4.3 (honest SKIP hardware_not_present), §11.4.69 (MEM evidence class), §11.4.107(1)/(10) (window-not-point liveness + self-validated analyzer), §11.4.50 (determinism), §11.4.81 (per-platform sampler), §11.4.108 (shipping artifact), §11.4.132 (G3/SLO4 high-risk), §11.4.118 (discovery boundary), §11.4.84 (quiescence), §11.4.66 (escalate G3 No-Go), §1.1 (paired mutation).